Structured Data with LLMs Done Right

A Practical Guide to Classification, Retrieval and Generation with LLMs

Jesus Villota [1]

CEMFI, Casado del Alisal 5, E-28014 Madrid, Spain

< jesus.villota@cemfi.edu.es >

Abstract

Economists increasingly use large language models (LLMs) to extract structured data from text, yet the standard approach of prompting a model to "return valid JSON" is unreliable, leading to format failures that corrupt observations. The solution, *Structured Output* schemas, constrain the model's token generation to follow a template defined ex-ante. We provide a practical guide for implementing this approach. We organize the methodology around three tasks that cover most text-as-data applications in economics: classification (e.g., labeling central bank statements as hawkish or dovish), information retrieval (e.g., extracting monetary policy decisions from FOMC meeting minutes), and structured generation (e.g., asking an LLM to give a loan rate based on a set of applicant characteristics). For each task, we provide schema design templates and implementation code compatible with major LLM providers. We demonstrate the practical implementation of these methods through a case study from Villota (2024), which extracts firm-level economic shocks from business news. To democratize this methodology, we designed SchemaForge, a website that allows researchers to select their model provider and design schemas interactively through a visual, block-based interface.

JEL Codes: C45, C55, C63, C80, C81, C82, C88

Keywords: Large Language Models, Natural Language Processing, Structured Output, Classification, Retrieval, Generation

[1] I am grateful to participants at the "New Methods" Bank of England Seminar for their feedback. I acknowledge financial support from the Santander Research Chair at CEMFI.

1. Introduction

Economists increasingly use large language models (LLMs) to extract structured data from text (Gentzkow et al., 2019, Korinek, 2023): classifying central bank statements as hawkish or dovish (Hansen and Kazinnik, 2024), extracting corporate risk exposures from earnings call transcripts (Kim et al., 2023), asking an LLM to give a loan rate based on a set of applicant characteristics (Bowen III et al., 2024). The standard approach is to write a natural language prompt instructing the model to return its output in a specified format, typically JSON. Consider a concrete example. A researcher wants to classify FOMC statements as hawkish, dovish, or neutral. The standard prompting approach looks like this:

LISTING 1. **Standard prompting approach for text classification**

```
prompt = f"""
Classify the monetary policy stance of the following FOMC statement.

Categories:
- "hawkish": Tighter monetary policy, concern about inflation
- "dovish": Looser monetary policy, concern about employment
- "neutral": Balanced assessment with no clear policy direction

Return the result in JSON format:
{{
  "classification": "hawkish",
  "confidence": 0.85,
  "reasoning": "Brief explanation"
}}

FOMC Statement:
"Recent indicators suggest that economic activity has continued
to expand at a solid pace. Inflation has eased over the past year
but remains somewhat elevated."

Provide your classification:
"""
```

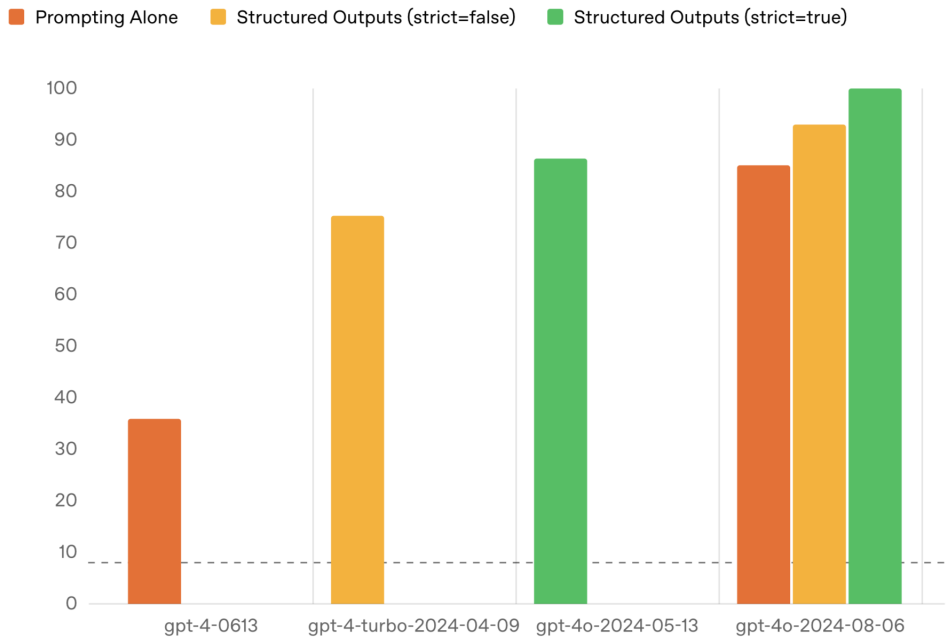
Nothing in this prompt *guarantees* that the model will return valid JSON, that the classification will be constrained to one of the three specified categories, or that the confidence score will be a number between 0 and 1. The same fragility applies to information retrieval tasks (extracting dates, amounts, and entities) and structured generation tasks (producing answers based on context). Across all three task types, the problem is

identical: the output format is a suggestion, not an enforcement.

According to OpenAI's internal evaluations on JSON schema following (OpenAI, 2024), prompting alone achieves only 35.9% adherence for `GPT-4-0613`, while `GPT-4o-2024-08-06` reaches 85.1% (Figure 1). In empirical research, an 85% success rate means that 15% of observations are corrupted, an error rate that no researcher would tolerate in other parts of their data pipeline. Even though new models may get better, JSON schema adherence cannot be fully enforced, and therefore guaranteed, under natural language prompting.

The failure is not in what the model knows but in how it is asked to respond. When a researcher prompts an LLM to "return JSON with fields X, Y, Z," the model is not *constrained* to follow that format; it is merely *instructed* to do so. The model may return text that is not valid JSON, use different key names, omit required fields, add unexpected ones, or return values outside the specified categories. Researchers then face bad choices: discard malformed responses (introducing selection bias), manually correct outputs (destroying reproducibility), or write brittle post-processing code that breaks with each model update.

FIGURE 1: **Model adherence to specific instructions.**



Note: Reliability metrics for how well different models adhere to a set of instructions when using prompting alone (red), enriching with a structured schema but without strict enforcement (yellow), and using *Structured Output* with strict schema enforcement (green). *Source:* OpenAI (2024)

The solution is *Structured Output*, a mechanism available from major LLM providers that allows researchers to define precise output schemas the model *must* follow. *Structured Output* constrain token generation at

inference time (Geng et al., 2023), converting output formatting from a probabilistic task into a deterministic guarantee. When strict schema enforcement is enabled, the model's final output is guaranteed to conform to the researcher's specification (Figure 1) while retaining full flexibility in its analytical decisions.

This paper provides a methodological guide for implementing structured data extraction from LLMs. We organize the discussion around three tasks that cover most use cases in empirical economics: text classification (assigning labels from a predefined set), information retrieval (extracting specific information from text), and structured generation (producing formatted outputs based on some textual context). For each task, we present schema design templates and implementation guidance drawn from industry best practices.

We demonstrate the framework through an empirical application from Villota (2024) analyzing Spanish firm-level shocks, while providing additional templates for common tasks in economics and finance research in the Appendix. To facilitate adoption, we have built `SchemaForge`, a web interface that allows researchers to design schemas interactively using a visual, block-based interface without requiring manual schema coding. Users select their model provider and export implementation code compatible with major LLM APIs.

The rest of the paper proceeds as follows. Section 2 presents the *Structured Output* framework and task-specific schema designs. Section 3 demonstrates the approach through a comprehensive empirical application. Section 4 concludes. Additional templates for common research tasks are provided in the Appendix.

2. Methodology

2.1 *Structured Output*

Structured Output is a feature, now available from major LLM providers including OpenAI, Anthropic, Google, Mistral, Meta, xAI, and others, that constrains model responses to conform to a researcher-supplied JSON Schema. Instead of prompting an LLM to format its response in a particular way and hope that it will comply, the researcher supplies a schema that the model *must* follow. The underlying mechanism, known as constrained decoding, restricts the set of tokens the model can sample at each generation step to only those that produce valid output under that schema: no missing fields, no invalid types, no malformed JSON.

Two Methods for Obtaining Structured Data Providers expose *Structured Output* through two distinct API interfaces. Both accept a JSON Schema and, when strict mode is enabled, both enforce it through constrained decoding. They differ in purpose and in where the structured result appears.

Response formatting is the first and most direct method. The researcher passes a JSON Schema via a response format parameter, and the model's text response is constrained to conform to that schema. The

result appears in the message content, exactly as if the model had written free-form text, except the text is guaranteed to be valid JSON matching the schema. This is the natural fit for data extraction: the schema defines the structure, and the model fills it from the input text. The parameter name varies by provider (OpenAI uses `response_format`; Anthropic uses `output_config.format`; Google Gemini uses `response_mime_type` with `response_json_schema`), but the underlying JSON Schema is identical.

Tool use (also called function calling) is the second method. The researcher defines "tools" with JSON Schema input schemas via the `tools` parameter. The model generates structured arguments as if calling an external function, and the result appears in `tool_calls` rather than in the message content. Tool use was originally designed for agentic workflows where the model triggers external functions (database queries, API calls, calculations), but it can also be repurposed for structured extraction. When strict mode is enabled (`strict: true`), tool use applies the same constrained decoding as response formatting, so schema enforcement is identical.

Table 1 summarizes the key differences between response formatting and tool use. For the tasks in this paper, response formatting is the more natural choice and we use it throughout. The schema design principles apply equally to both methods; only the API wrapper differs.

TABLE 1: **Response formatting vs. tool use for *Structured Output***

	Response formatting	**Tool use (function calling)**
Purpose	Structure the model's response to the user	Provide structured arguments to external functions
Schema enforcement	Constrained decoding (strict mode)	Constrained decoding (strict mode)
Where result appears	Message content	Tool calls
API parameter	`response_format` or equivalent	`tools` array
Primary use case	Data extraction, classification, generation	Agentic workflows; also usable for extraction

Two ways of writing schemas: JSON Schema and Pydantic Both response formatting and tool use accept a standard JSON Schema, which the researcher can write in two ways. The first is to write the schema directly as a JSON dictionary, specifying field names, types, descriptions, and constraints. This is the format shown in the code listings throughout this paper and is portable across all providers.

The second is to define the schema as a Pydantic `BaseModel` class in Python. Each class declares typed fields with optional descriptions; Pydantic converts the class definition into the equivalent JSON Schema automatically. The benefit is practical: schemas are easier to read, modify, and version-control as Python code, and

the returned JSON is parsed directly into a typed Python object with attribute access rather than a nested dictionary. Several provider SDKs accept Pydantic models directly in the API call: OpenAI's `client.chat.completions.par` Anthropic's `client.messages.parse()`, Groq's `client.chat.completions.parse()`, and Mistral's `client.chat.` all take a Pydantic class as the `response_format` argument, handle the JSON Schema conversion internally, and return a parsed object. For providers without native Pydantic support (Google Gemini, xAI, LLama), researchers can call `Model.model_json_schema()` to export the JSON Schema and pass it through the standard `response_format` dictionary.

2.2 Three Core Tasks

We define three core tasks that together cover the structured data needs of most empirical economics applications, before presenting the *Structured Output* framework and task-specific schema templates.

First, **text classification** assigns each text unit (a sentence, paragraph, or document) to one of a predefined set of categories. The researcher specifies the label space and the criteria for assignment; the model returns exactly one label per input. Typical applications include sentiment scoring of earnings calls, topic labeling of news articles, and regulatory document categorization.

Second, **information retrieval** extracts specific data fields or entities from text and returns them as a structured record. Each field has a name, a type (string, number, date, boolean), and instructions describing what to extract. This includes extracting named entities, dates, numerical values (amounts, metrics, counts), and identifiers, from unstructured sources such as filings, contracts, minutes, or reports.

Finally, **structured generation** produces new text that conforms to a predefined schema. Applications include generating standardized summaries, formatted reports, or metadata records. Economists can also use this for experiments: prompting LLMs to make decisions and analyzing their behavior to study decision-making patterns or biases.

All three tasks share a common schema structure. The schema defines what information the LLM must return and in what format while the LLM determines the actual content based on its analysis of the input text. The generic template is:

LISTING 2. **Unified *Structured Output* Schema Template**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "schema_name",
        "strict": true,  # Enforce 100% schema adherence
```

```
        "schema": {
            "type": "object",
            "properties": {
                # Define output fields here (task-specific)
                "field_1": {
                    "type": "type_1",
                    "description": "Instructions for field_1"
                },
                "field_2": {
                    "type": "type_2",
                    "description": "Instructions for field_2"
                },
                # Add more fields as needed
            },
            "required": ["field_1", "field_2"],
            "additionalProperties": false
        }
    }
}
```

The schema has five components. The `name` field is a descriptive label for the schema (e.g., `classify_sentiment`, `extract_financial_data`). The `strict` flag, set to `true`, activates constrained decoding: OpenAI and Anthropic expose this as an explicit parameter (OpenAI for both response formatting and function calling; Anthropic for tool use, with implicit enforcement for JSON outputs), while providers such as Google Gemini enforce schemas implicitly whenever *Structured Output* are enabled. The underlying mechanism is identical across providers regardless of API syntax. The `properties` object defines the output fields; each field carries a `type` (`string`, `number`, `integer`, `boolean`, `array`, or `object`), a `description` with instructions for how the model should determine the field's value, and an optional `enum` for fields with a finite set of valid values. The `required` array lists the fields that must always appear in the output. Finally, `additionalProperties` set to `false` prevents the model from returning fields not declared in the schema.

The same schema structure applies to classification (where properties define categories and constraints), retrieval (where properties define fields to extract), and generation (where properties define components to produce). 3 shows the Pydantic equivalent of the unified schema template in Listing 2:

LISTING 3. **Pydantic Equivalent of the Unified Schema Template**

```python
from pydantic import BaseModel, Field

class SchemaName(BaseModel):
    field_1: type_1 = Field(
        description="Instructions for field_1"
    )
    field_2: type_2 = Field(
        description="Instructions for field_2"
    )
    # Add more fields as needed
```

2.3 Task-Specific Schema Adaptations

While the unified template in Listing 2 provides the common structure, the three tasks differ in which JSON Schema features do the real work: `enum` constraints for classification, typed field definitions for information retrieval, and mixed output fields with constrained choices for generation. We present a template for each below.

2.3.1 Text Classification Schema

For classification tasks, the key element is the `enum` constraint, which restricts the model to selecting from a predefined set of categories.

LISTING 4. **Classification Schema Template**

```python
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "classifyer",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "classification": {
                    "type": "string",  # or integer, number, boolean
                    "enum": [c_1, c_2, ..., c_k],
```

```
                    "description": "Detailed classification instructions"
                },
                "confidence": {
                    "type": "number",
                    "minimum": 0,
                    "maximum": 1,
                    "description": "State a confidence score (between 0 and 1) for your classification"
                },
                "reasoning": {
                    "type": "string",
                    "description": "Explain your classification decision"
                }
            },
            "required": ["classification", "confidence", "reasoning"],
            "additionalProperties": false
        }
    }
}
```

The classification field type should match the nature of the categories. Table 2 summarizes the available types: `string` for named categories (e.g., sentiment labels), `integer` for ordinal scales, `number` for discrete rating grids, and `boolean` for binary decisions.

TABLE 2: **Classification field types and examples**

Type	Use Case	Example Enum Values
`string`	Named categories	["positive", "negative", "neutral"]
`integer`	Ordinal scales	[-2, -1, 0, 1, 2]
`number`	Continuous ratings	[-1.0, -0.5, 0.0, 0.5, 1.0]
`boolean`	Binary classification	[true, false]

The `description` field carries the classification logic. Write them as instructions for a research assistant: provide clear discriminating criteria for each category, include prototypical examples where helpful, and specify how to handle edge cases and ambiguity.

2.3.2 Information Retrieval Schema

For information retrieval, the schema defines multiple fields to extract, each with its own type and instructions.

LISTING 5. **Information Retrieval Schema Template**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "extractor",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "entity_name": {
                    "type": "string",
                    "description": "Name or identifier of the primary entity"
                },
                "date": {
                    "type": ["string", "null"],
                    "description": "Date in YYYY-MM-DD format.  Use null if not found."
                },
                "amount": {
                    "type": "number",
                    "description": "Numerical value (e.g., transaction amount, forecast)"
                },
                "is_verified": {
                    "type": "boolean",
                    "description": "Whether the information is explicitly stated vs. inferred"
                },
                "related_entities_list": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "List of related entities (e.g., companies, people, locations)"
                },
                "confidence": {
                    "type": "number",
                    "minimum": 0,
```

```
                    "maximum": 1,
                    "description": "Confidence in the extracted information (0 to 1)"
                },
                "reasoning": {
                    "type": "string",
                    "description": "Explain how the information was identified and extracted"
                }
            },
            "required": ["entity_name", "date", "amount", "is_verified",
    "related_entities_list", "confidence", "reasoning"],
            "additionalProperties": false
        }
    }
}
```

Table 3 maps common data types to their JSON Schema equivalents. Strings cover names, descriptions, and dates (with a consistent format such as YYYY-MM-DD specified in the description); numbers and integers handle financial values and discrete counts; booleans capture binary flags; and arrays and nested objects accommodate lists of entities or hierarchical records.

TABLE 3: **Field types for information retrieval**

Data Type	JSON Type	Usage Notes
Names, text	`string`	Entity names, descriptions
Dates	`string`	Use consistent format (YYYY-MM-DD)
Amounts, metrics	`number`	Financial values, percentages, rates
Counts, rankings	`integer`	Discrete quantities
Yes/No flags	`boolean`	Binary properties
Lists of items	`array`	Multiple entities, keywords, topics
Nested structures	`object`	Complex hierarchical data

Not all fields will be present in every text. With strict mode enabled, all fields must appear in the `required` array, as the model cannot simply omit a field. To handle missing information, mark the field as nullable by setting its type to `["type", "null"]` (e.g., `"type": ["string", "null"]`, as illustrated by the `date` field in Listing 5), and specify in the field's description how the model should respond when the information is absent (e.g., "Use null if not mentioned"). For array fields, an empty array is preferable to null since the schema type remains `array`. For `enum` fields, an explicit `not_found` or `not_applicable` value gives the

model a valid option when the category does not apply, avoiding the need for nullability altogether.

2.3.3 Structured Generation Schema

For generation tasks, the schema defines what the model must produce: a primary response or decision, any associated numerical outcomes, and supporting justifications.

LISTING 6. **Structured Generation Schema Template**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "generator",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "decision": {
                    "type": "object",
                    "properties": {
                        "choice": {
                            "type": "string",
                            "enum": ["option_A", "option_B", "option_C"],
                            "description": "Primary choice or action selected by the model"
                        },
                        "sub_choice": {
                            "type": "string",
                            "enum": ["sub_1", "sub_2", "sub_3"],
                            "description": "Follow-up choice conditional on the primary
    decision"
                        }
                    },
                    "required": ["choice", "sub_choice"],
                    "additionalProperties": false,
                    "description": "Decision tree:  primary choice and dependent follow-up"
                },
                "outcome": {
                    "type": "number",
                    "description": "Numerical result associated with the decision"
                },
```

```
                "explanation": {
                    "type": "string",
                    "description": "Free-text justification for the decision"
                },
                "reasoning": {
                    "type": "string",
                    "description": "Step-by-step reasoning behind the generated output"
                }
            },
            "required": ["decision", "outcome", "explanation", "reasoning"],
            "additionalProperties": false
        }
    }
}
```

The `decision` field is a nested object that groups a primary `choice` with a dependent `sub_choice`, enforcing a decision-tree structure directly through the schema. Both use `enum` constraints to restrict the model to valid options at each stage; the dependency between them (which sub-choices are valid given the primary choice) is specified in the `description` fields, where it functions as instructions to the model. The `outcome` field captures any numerical quantity the model must report alongside its decision (a score, amount, probability, or count), while `explanation` collects the model's free-text justification. When a task requires additional sub-components or lists within the output, arrays of objects and `minItems`/`maxItems` constraints can be added following the same principles as the unified template.

2.4 Implementation and API Integration

As discussed in Section 2.1, the two methods for obtaining structured data differ only in how the schema is passed to the API and where the result appears. We use response formatting throughout this paper, passing the schema via the `response_format` parameter. The syntax below follows the OpenAI API, which is also compatible with providers that adopt the OpenAI API specification (e.g: GroqCloud).

LISTING 7. **API Call with *Structured Output* (response_format)**

```
response = client.chat.completions.create(
    model="model-name",
    messages=[
        {"role": "system", "content": "System instructions"},
```

```
        {"role": "user", "content": "User prompt with text to analyze"}
    ],
    response_format=response_format,  # The schema defined above
    temperature=0.0  # Lower for consistency
)

# Extract the structured result
result = json.loads(response.choices[0].message.content)
```

Listing 7 passes a raw JSON Schema dictionary and parses the response with `json.loads`. When the schema is defined as a Pydantic class instead (Section 2.1), provider SDKs offer a `parse()` method that accepts the class directly as `response_format`, converts it to JSON Schema internally, and returns a typed object. Listing 8 shows this alternative using OpenAI's `client.chat.completions.parse()`, which is also available for GroqCloud and, with minor syntax differences, for Anthropic (`client.messages.parse()`) and Mistral (`client.chat.parse()`). The `message.parsed` attribute returns a typed Pydantic object with attribute access, and `message.refusal` exposes safety refusals when the model declines to respond.

LISTING 8. **API Call with Pydantic SDK Parse (`response_format`)**

```
completion = client.chat.completions.parse(
    model="model-name",
    messages=[
        {"role": "system", "content": "System instructions"},
        {"role": "user", "content": "User prompt with text to analyze"}
    ],
    response_format=PydanticSchema,  # Pydantic class
    temperature=0.0
)

message = completion.choices[0].message
if message.refusal:
    print(f"Model refused: {message.refusal}")
else:
    result = message.parsed  # Typed Pydantic object
```

Researchers who prefer the tool use interface (or whose provider requires it) pass the same JSON Schema wrapped in a `tools` array with `"type": "function"`, as shown in Listing 9. Each tool definition includes a `name`, a `description`, and a `parameters` object containing the schema. With `strict: true`, constrained

decoding applies identically to response formatting. The structured result appears in `tool_calls` rather than `message.content`: the raw JSON string is available via `function.arguments`, while SDKs that support Pydantic return a typed object through `function.parsed_arguments`. OpenAI's `openai.pydantic_function_tool()` helper converts a Pydantic class into the required tool definition dictionary automatically.

LISTING 9. **API Call with Tool Use (Function Calling)**

```python
tools = [{
    "type": "function",
    "function": {
        "name": "extract_data",
        "description": "Extract structured data from text.",
        "parameters": response_format["json_schema"]["schema"],
        "strict": true
    }
}]

response = client.chat.completions.create(
    model="model-name",
    messages=[
        {"role": "system", "content": "System instructions"},
        {"role": "user", "content": "User prompt with text to analyze"}
    ],
    tools=tools,
    temperature=0.0
)

# Extract from tool_calls instead of message.content
result = json.loads(
    response.choices[0].message.tool_calls[0].function.arguments
)
```

The schema design principles in this paper apply identically regardless of which interface is used; only the API wrapper differs. We use response formatting throughout because it is the more natural fit for data extraction, but tool use is functionally equivalent when strict mode is enabled.

2.5 Implementation Best Practices

Before processing an entire corpus, we recommend testing the schema on a small labeled subset of 30 to 50 examples to validate extraction accuracy and refine field descriptions. The `reasoning` field is useful during this development phase: inspecting the model's stated rationale for each classification decision reveals recurrent misinterpretations that can be corrected by adjusting descriptions or adding disambiguating examples. If possible, validating a random sample of 10–20% of model outputs against independent human coding provides a direct measure of inter-rater reliability and exposes error patterns that pilot testing alone may miss.

Model choice matters. Different architectures vary in their accuracy across extraction, classification, and generation tasks, so researchers should benchmark several candidates on their validation set and select based on accuracy-cost tradeoffs specific to the task at hand. Temperature configuration follows a simple rule: set `temperature=0.0` for classification and extraction tasks where consistency is the priority, and consider values in the 0.3 to 0.5 range only for generation tasks where some output variation is acceptable. When reproducibility is critical, consider setting a fixed `seed` parameter when the API provider supports it.

On the operational side, most API providers offer batch endpoints that reduce both per-token costs and latency relative to sequential single-request calls. Before committing to a full production run, estimating total costs from token counts avoids budget surprises. Using smaller models such as Llama-3-8b or GPT-4o-mini for initial development and debugging further controls costs during the iterative design phase.

Reproducibility requires documenting the exact model version used (e.g., `llama3-70b-8192`), the complete schema, all system prompts, and every parameter setting, including temperature and any post-processing rules. Replication packages should include the full schema definitions, prompt templates, parameter configurations, and example API calls, so that other researchers can reproduce the extraction pipeline without ambiguity.

3. Empirical Application: Predicting Market Reactions to News

We demonstrate the *Structured Output* methodology through a complete example from Villota (2024), which extracts firm-specific economic shocks from Spanish business news articles to quantify their impact on stock returns. The application requires the LLM to simultaneously identify relevant entities (publicly traded Spanish firms) and classify the nature of the news affecting them, translating unstructured daily articles into a structured dataset of news-implied firm-specific shocks. For every article, the model identifies all publicly listed Spanish firms directly affected by the news and, for each, classifies the shock along three dimensions: its *type* (demand, supply, financial, policy, or technology), its *magnitude* (minor or major), and its *direction*

(positive or negative). This task is particularly challenging because a single article might discuss multiple firms experiencing different types of shocks simultaneously; a simple text classification approach that labels the entire article would fail to capture firm-specific heterogeneity.

3.1 Schema Design

To guarantee that the model's output can be directly parsed into a dataframe without human intervention, we define a strict JSON Schema passed via the `response_format` parameter. Strict mode requires all fields to appear in the `required` array and all objects to set `additionalProperties=false`, which together ensure 100% schema adherence through constrained decoding.

LISTING 10. **News Parsing Schema for Firm-Level Shocks**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "firm_shock_parser",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "firms": {
                    "type": "array",
                    "description": "List the Spanish firms directly affected by the news. Firms must be publicly listed on the Spanish stock exchange (ticker format: TICKER.MC). Exclude foreign firms, unlisted firms, and firms mentioned only for context.",
                    "items": {
                        "type": "object",
                        "properties": {
                            "firm": {
                                "type": "string",
                                "description": "Name of the Spanish firm directly affected by the news."
                            },
                            "ticker": {
                                "type": "string",
                                "description": "Yahoo Finance ticker (e.g., ITX.MC for Inditex, SAN.MC for Banco Santander)."
                            },
```

```
                            "shock_type": {
                                "type": "string",
                                "enum": ["demand", "supply", "financial", "policy", "technology"],
                                "description": "Type of shock: 'demand' for consumer demand events, 'supply' for supply-chain events, 'financial' for market conditions, 'policy' for regulatory changes, 'technology' for tech disruptions."
                            },
                            "shock_magnitude": {
                                "type": "string",
                                "enum": ["minor", "major"],
                                "description": "Expected strength: 'minor' or 'major'."
                            },
                            "shock_direction": {
                                "type": "string",
                                "enum": ["positive", "negative"],
                                "description": "Expected direction: 'positive' for beneficial impacts, 'negative' for adverse impacts."
                            }
                        },
                        "required": ["firm", "ticker", "shock_type",
                                     "shock_magnitude", "shock_direction"],
                        "additionalProperties": false
                    }
                }
            },
            "required": ["firms"],
            "additionalProperties": false
        }
    }
}
```

Listing 11 shows the equivalent schema defined using Pydantic, which can be passed directly to providers that support SDK-level parsing (e.g., `client.chat.completions.parse()` for OpenAI and GroqCloud).

LISTING 11. **Pydantic Equivalent of the News Parsing Schema**

```
from pydantic import BaseModel, Field
from typing import Literal

```

```python
class FirmShock(BaseModel):
    firm: str = Field(
        description="Name of the Spanish firm directly affected by the news."
    )
    ticker: str = Field(
        description="Yahoo Finance ticker (e.g., ITX.MC for Inditex, SAN.MC for Banco Santander)."
    )
    shock_type: Literal["demand", "supply", "financial", "policy", "technology"] = Field(
        description="Type of shock: 'demand' for consumer demand events, 'supply' for supply-chain events, 'financial' for market conditions, 'policy' for regulatory changes, 'technology' for tech disruptions."
    )
    shock_magnitude: Literal["minor", "major"] = Field(
        description="Expected strength: 'minor' or 'major'."
    )
    shock_direction: Literal["positive", "negative"] = Field(
        description="Expected direction: 'positive' for beneficial, 'negative' for adverse."
    )

class FirmShockParser(BaseModel):
    firms: list[FirmShock] = Field(
        description="List of Spanish firms directly affected by the news. Exclude foreign firms, unlisted firms, and firms mentioned only for context."
    )
```

3.2 Inner Functionings of the Schema

The schema dictates both the hierarchical structure of the data and the permissible values for each field. By defining the root `firms` property as an `array` of `objects`, the schema natively supports articles that discuss zero, one, or multiple firms. If an article discusses a merger between two listed Spanish firms, the model generates an array containing two objects, capturing the (potentially differing) shocks for each firm. If the article contains no relevant shocks for listed Spanish firms, the model simply returns an empty array `[]`.

The classification dimensions (`shock_type`, `shock_magnitude`, `shock_direction`) are strictly constrained using `enum` arrays. By restricting `shock_direction` to strictly `["positive", "negative"]`, and omitting a "neutral" option, the schema structurally enforces the instruction: *If a firm is neutral to the article, do NOT include it in the analysis.* The model cannot return a neutral shock because it is not a topologically valid output under the schema.

The `description` fields serve as highly targeted prompts attached directly to the data structures they govern. For example, the description for the `ticker` field explicitly reminds the model of the required formatting (`TICKER.MC`) and provides prototypical examples (`ITX.MC`, `SAN.MC`). This significantly improves extraction accuracy compared to placing all instructions in a monolithic system prompt.

Each field in the schema maps to a particular task as defined in Section 2.1. Table 4 makes this correspondence explicit. The schema simultaneously performs two distinct task types in a single API call: *information retrieval* for firm-ticker identification and *classification* for categorizing the firm-specific nature of each shock.

TABLE 4: **Mapping schema fields to core tasks**

Schema Field	**Task Type**	**Mechanism**
`firms` (array of objects)	Information Retrieval	Entity extraction; the array structure handles zero, one, or many affected firms per article
`firm` (string)	Information Retrieval	Free-text extraction of the firm's name from the article
`ticker` (string)	Information Retrieval	Factual knowledge retrieval of the Yahoo Finance identifier (`TICKER.MC` format)
`shock_type` (string, enum)	Classification	5-class classification constrained to `demand`, `supply`, `financial`, `policy`, `technology`
`shock_magnitude` (string, enum)	Classification	Binary ordinal classification: `minor` vs. `major`
`shock_direction` (string, enum)	Classification	Binary classification: `positive` vs. `negative`; absence of a "neutral" option structurally enforces the filtering instruction

The retrieval fields (`firm`, `ticker`) are unconstrained strings whose values depend on the article content and the model's knowledge. The classification fields (`shock_type`, `shock_magnitude`, `shock_direction`) use `enum` constraints to restrict output to a predefined label space. This distinction matters because classification fields benefit directly from strict mode's constrained decoding: the model can only sample tokens that produce valid enum values, eliminating misclassification due to formatting errors entirely.

3.3 Implementation with *Structured Output*

To execute the extraction, the schema from Listing 10 is passed to the LLM along with a system prompt and the article text via the `response_format` parameter. Listing 12 shows the raw JSON Schema approach using Llama-3-70b on the Groq API. The response is returned as a JSON string in `message.content`, which `json.loads` parses directly into a Python dictionary.

LISTING 12. **Executing the Extraction via** `response_format`

```python
import json
from groq import Groq

client = Groq(api_key=api_key)

response = client.chat.completions.create(
    model="llama3-70b-8192",
    temperature=0.0,
    response_format=response_format,  # Schema from Listing 1
    messages=[
        {"role": "system", "content": "You are a helpful assistant "
         "that analyses business news in Spanish..."},
        {"role": "user",   "content": article_text},
    ],
)

# Parse the structured output from message content
result = json.loads(response.choices[0].message.content)
extracted_firms = result.get("firms", [])

for firm_data in extracted_firms:
    record = {
        "firm":       firm_data["firm"],
        "ticker":     firm_data["ticker"],
        "shock_type": firm_data["shock_type"],
        "magnitude":  firm_data["shock_magnitude"],
        "direction":  firm_data["shock_direction"],
    }
    # Append record to dataset
```

Listing 13 shows the equivalent implementation using the Pydantic SDK Parse method. The `parse()` method accepts a Pydantic class as `response_format`, handles JSON Schema conversion and validation internally, and returns a typed object with attribute access rather than raw dictionaries.

LISTING 13. **Pydantic SDK Parse Equivalent**

```python
from groq import Groq

client = Groq(api_key=api_key)

completion = client.chat.completions.parse(
    model="llama3-70b-8192",
    temperature=0.0,
    response_format=FirmShockParser,  # Pydantic class from Listing 2
    messages=[
        {"role": "system", "content": "You are a helpful assistant "
         "that analyses business news in Spanish..."},
        {"role": "user",   "content": article_text},
    ],
)

message = completion.choices[0].message
if message.refusal:
    print(f"Model refused: {message.refusal}")
else:
    result = message.parsed  # Typed FirmShockParser object
    for firm_data in result.firms:
        record = {
            "firm":       firm_data.firm,
            "ticker":     firm_data.ticker,
            "shock_type": firm_data.shock_type,
            "magnitude":  firm_data.shock_magnitude,
            "direction":  firm_data.shock_direction,
        }
        # Append record to dataset
```

Because strict mode guarantees that every response conforms to the schema through constrained decoding, the parsing code in both listings will not throw errors. Dictionary lookups in Listing 12 and attribute access in Listing 13 yield clean, standardized categorical variables ready for regression analysis. The `enum` constraints on classification fields mean that `shock_type`, `shock_magnitude`, and `shock_direction` contain only valid category labels, eliminating the need for post-processing or string normalization.

4. Conclusion

The standard approach to extracting structured data from text using large language models relies on natural language instructions: researchers prompt the model to return output in a specified format, typically JSON. This approach is fragile because format compliance is a suggestion, not a constraint. The model may return malformed JSON, use unexpected field names, omit required fields, or return values outside the specified categories. Researchers then face a bad choice: discard malformed responses and introduce selection bias, manually correct outputs and destroy reproducibility, or write brittle post-processing code that breaks with each model update.

Structured Output schemas resolve this by shifting format compliance from the prompt to the model's generation mechanism. Through constrained decoding, the model is restricted at inference time to only those tokens that produce valid output under the researcher-supplied schema. The output format is thereby guaranteed regardless of the model's analytical decisions. We present a unified schema framework covering three tasks that together span most text-as-data applications in economics: text classification, information retrieval, and structured generation. The framework is compatible with all major LLM providers and requires only minimal adaptation to specific research contexts.

We demonstrate the approach through a complete empirical application drawn from Villota (2024), which extracts firm-level economic shocks from Spanish business news. Each article is processed by the model to identify affected publicly listed firms, retrieve their ticker symbols, and classify the associated shock along three dimensions: type, magnitude, and direction. The schema enforces both the hierarchical structure of the output and the permissible values for each field, producing a dataset ready for downstream regression analysis without manual intervention.

To facilitate adoption, we have built `SchemaForge`, a web interface that allows researchers to design schemas interactively through a visual, block-based interface without writing raw JSON Schema by hand. Users select their model provider and export implementation code compatible with major LLM APIs. Ready-to-use templates for common tasks in economics and finance research are provided in the Appendix.

References

D. E. Bowen III, S. M. Price, L. C. Stein, and K. Yang. Measuring and mitigating racial disparities in large language model mortgage underwriting. Working paper, SSRN, 2024. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4812158.

S. Geng, M. Josifoski, M. Peyrard, and R. West. Grammar-constrained decoding for structured NLP tasks without

finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.674.

M. Gentzkow, B. Kelly, and M. Taddy. Text as data. *Journal of Economic Literature*, 57(3):535–574, September 2019. doi: 10.1257/jel.20181020.

A. L. Hansen and S. Kazinnik. Can ChatGPT decipher Fedspeak? Working paper, SSRN, 2024. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4399406.

A. Kim, M. Muhn, and V. Nikolaev. From transcripts to insights: Uncovering corporate risks using generative AI. Working paper, University of Chicago Booth School of Business, 2023. URL https://arxiv.org/abs/2310.17721.

A. Korinek. Generative AI for economic research: Use cases and implications for economists. *Journal of Economic Literature*, 61(4):1281–1317, 2023. doi: 10.1257/jel.20231736.

OpenAI. Introducing structured outputs in the API. Blog post, August 2024. URL https://openai.com/index/introducing-structured-outputs-in-the-api/. Accessed: 2025.

J. Villota. Predicting market reactions to news: An llm-based approach using spanish business articles. Working Paper 2501, CEMFI, June 2024. URL https://ssrn.com/abstract=5006857.

A. Appendix: Empirical Applications Templates

This appendix provides ready-to-use structured output schemas for practical economic research applications. We organize examples by task type (classification, retrieval, and generation) before showing how to combine multiple tasks in complex research workflows. Each subsection provides complete, ready-to-use templates that researchers can adapt to their specific use cases.

A.1 Classification Applications

These schemas assign texts to predefined categories using `enum`-constrained fields.

A.1.1 Central Bank Communication Analysis

This schema classifies monetary policy statements, speeches, and meeting minutes as hawkish, dovish, or neutral.

LISTING 14. **Central Bank Communication Classification**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "classify_monetary_policy_stance",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "policy_stance": {
                    "type": "string",
                    "enum": ["hawkish", "dovish", "neutral"],
                    "description": """Classify the text as:
                    - 'hawkish': preference for raising interest rates,
                      concerns about inflation, tightening monetary policy
                    - 'dovish': preference for lowering rates, concerns about
                      unemployment/growth, easing monetary policy
                    - 'neutral': balanced view or no clear policy direction"""
                },
                "confidence": {
                    "type": "number",
```

```
                    "description": "Confidence score for the classification (0 to 1)"
                },
                "reasoning": {
                    "type": "string",
                    "description": "Brief explanation of the classification"
                }
            },
            "required": ["policy_stance", "confidence", "reasoning"],
            "additionalProperties": false
        }
    }
}
```

A.1.2 Financial News Sentiment Analysis

This schema classifies financial news articles by sentiment direction and intensity.

LISTING 15. **Financial Sentiment Classification**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "classify_financial_sentiment",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "sentiment": {
                    "type": "string",
                    "enum": ["positive", "negative", "neutral"],
                    "description": """Classify overall sentiment:
                    - 'positive': favorable conditions, positive developments,
                      bullish outlook
                    - 'negative': unfavorable conditions, negative developments,
                      bearish outlook
                    - 'neutral': balanced or factual without clear implications"""
                },
                "intensity": {
                    "type": "integer",
```

```
                    "enum": [1, 2, 3],
                    "description": "Intensity: 1=mild, 2=moderate, 3=strong"
                },
                "confidence": {
                    "type": "number",
                    "description": "Confidence score (0 to 1)"
                },
                "reasoning": {
                    "type": "string",
                    "description": "Key phrases supporting classification"
                }
            },
            "required": ["sentiment", "intensity", "confidence", "reasoning"],
            "additionalProperties": false
        }
    }
}
```

A.1.3 ESG Classification

This schema classifies corporate disclosures, annual reports, and press releases by primary ESG dimension and commitment level.

LISTING 16. **ESG Dimension Classification**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "classify_esg_dimension",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "primary_dimension": {
                    "type": "string",
                    "enum": ["environmental", "social", "governance", "none"],
                    "description": """Primary ESG focus:
                    - 'environmental': climate, emissions, resource use, pollution
                    - 'social': labor, human rights, community, diversity
```

```
                    - 'governance': board structure, compensation, ethics
                    - 'none': no significant ESG content"""
                },
                "commitment_level": {
                    "type": "string",
                    "enum": ["aspirational", "committed", "implemented", "reported"],
                    "description": """ESG commitment level:
                    - 'aspirational': future goals
                    - 'committed': formal pledges
                    - 'implemented': actions being taken
                    - 'reported': results disclosed"""
                },
                "confidence": {
                    "type": "number",
                    "description": "Confidence score (0 to 1)"
                },
                "reasoning": {
                    "type": "string"
                }
            },
            "required": ["primary_dimension", "commitment_level", "confidence", "reasoning"],
            "additionalProperties": false
        }
    }
}
```

A.2 Retrieval Applications

These schemas extract specific data fields and entities from unstructured text.

A.2.1 Extracting Economic Indicators from Central Bank Communications

This schema extracts economic forecasts and policy rate decisions from central bank meeting minutes and monetary policy reports.

Listing 17. **Economic Indicator Extraction**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "extract_economic_indicators",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "gdp_forecast": {
                    "type": "number",
                    "description": "GDP growth forecast (percentage). Use null if not mentioned."
                },
                "inflation_forecast": {
                    "type": "number",
                    "description": "Inflation forecast (percentage). Use null if not mentioned."
                },
                "unemployment_forecast": {
                    "type": "number",
                    "description": "Unemployment rate forecast (percentage). Use null if not mentioned."
                },
                "policy_rate_change": {
                    "type": "number",
                    "description": "Policy rate change in basis points (e.g., 25 for 0.25%). Use 0 if no change."
                },
                "forecast_horizon": {
                    "type": "string",
                    "description": "Time horizon for forecasts (e.g., '2024 Q4', '2025', 'current year')"
                },
                "confidence": {
                    "type": "number",
                    "description": "Confidence in the extracted information (0 to 1)"
                }
            },
```

```
            "required": ["policy_rate_change", "forecast_horizon", "confidence"],
            "additionalProperties": false
        }
    }
}
```

A.2.2 Financial Contract Information Extraction

This schema extracts loan terms, counterparties, and collateral information from financial contracts and loan agreements.

LISTING 18. **Loan Contract Information Extraction**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "extract_loan_terms",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "loan_amount": {
                    "type": "number",
                    "description": "Principal loan amount in dollars"
                },
                "interest_rate": {
                    "type": "number",
                    "description": "Annual interest rate (percentage)"
                },
                "maturity_date": {
                    "type": "string",
                    "description": "Loan maturity date (YYYY-MM-DD format)"
                },
                "collateral_type": {
                    "type": "string",
                    "enum": ["real_estate", "equipment", "inventory", "receivables",
                             "securities", "unsecured", "other"],
                    "description": "Type of collateral securing the loan"
                },
```

```
                "borrower_entity": {
                    "type": "string",
                    "description": "Name of the borrowing entity"
                },
                "lender_entity": {
                    "type": "string",
                    "description": "Name of the lending institution"
                },
                "covenants_present": {
                    "type": "boolean",
                    "description": "Whether financial covenants are specified"
                },
                "confidence": {
                    "type": "number",
                    "description": "Confidence score (0 to 1)"
                }
            },
            "required": ["loan_amount", "interest_rate", "borrower_entity",
                         "lender_entity", "confidence"],
            "additionalProperties": false
        }
    }
}
```

A.2.3 Corporate Event Extraction from News

This schema extracts corporate event details, including event type, companies involved, and transaction values, from news articles.

LISTING 19. **Corporate Event Extraction**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "extract_corporate_event",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
```

```
                "event_type": {
                    "type": "string",
                    "enum": ["merger", "acquisition", "earnings", "dividend",
                             "executive_change", "product_launch", "lawsuit",
                             "regulatory", "other"],
                    "description": "Type of corporate event"
                },
                "event_date": {
                    "type": "string",
                    "description": "Event date (YYYY-MM-DD). Use article date if event date not specified."
                },
                "companies_involved": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "List of company names involved in the event"
                },
                "transaction_value": {
                    "type": "number",
                    "description": "Monetary value in millions USD. Use null if not applicable."
                },
                "market_reaction": {
                    "type": "string",
                    "enum": ["positive", "negative", "neutral", "mixed", "not_mentioned"],
                    "description": "Described market reaction to the event"
                },
                "confidence": {
                    "type": "number",
                    "description": "Confidence score (0 to 1)"
                }
            },
            "required": ["event_type", "event_date", "companies_involved", "confidence"],
            "additionalProperties": false
        }
    }
}
```

A.2.4 Research Paper Metadata Extraction

This schema extracts publication metadata, methodology classification, and key findings from academic papers.

LISTING 20. **Research Paper Metadata Extraction**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "extract_paper_metadata",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "publication_year": {
                    "type": "integer",
                    "description": "Year of publication"
                },
                "methodology": {
                    "type": "string",
                    "enum": ["experimental", "quasi_experimental", "regression",
                             "time_series", "panel_data", "structural",
                             "machine_learning", "theoretical", "survey"],
                    "description": "Primary empirical methodology"
                },
                "data_period_start": {
                    "type": "string",
                    "description": "Start of data period (YYYY or YYYY-MM)"
                },
                "data_period_end": {
                    "type": "string",
                    "description": "End of data period (YYYY or YYYY-MM)"
                },
                "jel_codes": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "JEL classification codes"
                },
                "key_findings": {
                    "type": "array",
```

```
                    "items": {"type": "string"},
                    "description": "List of main findings (3-5 bullet points)",
                    "minItems": 1,
                    "maxItems": 5
                },
                "causal_inference": {
                    "type": "boolean",
                    "description": "Whether paper establishes causal relationships"
                }
            },
            "required": ["publication_year", "methodology", "jel_codes", "key_findings"],
            "additionalProperties": false
        }
    }
}
```

A.3 Generation Applications

These schemas generate new structured content from input text.

A.3.1 Policy Brief Generation from Legislation

This schema generates standardized policy briefs, including fiscal impact estimates and key provisions, from legislative documents.

LISTING 21. **Policy Brief Generation**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "generate_policy_brief",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "title": {
                    "type": "string",
                    "description": "Brief descriptive title (max 15 words)"
                },
```

```
                "executive_summary": {
                    "type": "string",
                    "description": "High-level summary for policymakers (max 150 words)"
                },
                "affected_sectors": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "Economic sectors affected by the policy",
                    "minItems": 1
                },
                "fiscal_impact": {
                    "type": "object",
                    "properties": {
                        "estimated_cost": {
                            "type": "number",
                            "description": "Estimated fiscal cost in millions USD"
                        },
                        "revenue_impact": {
                    "type": "number",
                            "description": "Revenue impact in millions USD (positive or negative)"
                        },
                        "time_horizon": {
                            "type": "string",
                            "description": "Time period for estimates (e.g., 'annual', '10-year')"
                        }
                    },
                    "required": ["time_horizon"],
                    "additionalProperties": false
                },
                "implementation_timeline": {
                    "type": "string",
                    "description": "Expected timeline for implementation"
                },
                "key_provisions": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "List of major provisions (3-7 items)",
                    "minItems": 3,
```

```
                    "maxItems": 7
                }
            },
            "required": ["title", "executive_summary", "affected_sectors",
                         "implementation_timeline", "key_provisions"],
            "additionalProperties": false
        }
    }
}
```

A.3.2 Company Profile Generation from Annual Reports

This schema generates standardized company profiles, including financial highlights, risk factors, and competitive positioning, from annual reports.

LISTING 22. **Company Profile Generation**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "generate_company_profile",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "company_overview": {
                    "type": "string",
                    "description": "Brief company description (max 100 words)"
                },
                "financial_highlights": {
                    "type": "object",
                    "properties": {
                        "revenue": {"type": "number", "description": "Revenue in millions"},
                        "net_income": {"type": "number"},
                        "total_assets": {"type": "number"},
                        "year": {"type": "integer"}
                    },
                    "required": ["year"],
                    "additionalProperties": false
```

```
                },
                "key_risks": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "Major risk factors (3-5 items)",
                    "minItems": 3,
                    "maxItems": 5
                },
                "strategic_priorities": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "Key strategic priorities (3-5 items)",
                    "minItems": 3,
                    "maxItems": 5
                },
                "competitive_position": {
                    "type": "string",
                    "enum": ["market_leader", "strong_competitor", "emerging_player",
                             "niche_player", "challenged"],
                    "description": "Assessment of competitive position"
                }
            },
            "required": ["company_overview", "financial_highlights", "key_risks",
                         "strategic_priorities"],
            "additionalProperties": false
        }
    }
}
```

A.3.3 Literature Review Summary Generation

This schema generates structured literature review summaries, including research questions, methodology, findings, and limitations, from academic papers.

LISTING 23. **Literature Review Summary**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
```

```
        "name": "generate_literature_summary",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                "main_research_question": {
                    "type": "string",
                    "description": "The central research question (one sentence)"
                },
                "methodology_summary": {
                    "type": "string",
                    "description": "Brief methodology description (max 100 words)"
                },
                "data_description": {
                    "type": "string",
                    "description": "Data sources and sample period (max 80 words)"
                },
                "key_findings": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "Main findings (3-5 bullet points)",
                    "minItems": 3,
                    "maxItems": 5
                },
                "contribution": {
                    "type": "string",
                    "description": "Main contribution to literature (max 100 words)"
                },
                "limitations": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "Key limitations or caveats (2-4 items)",
                    "minItems": 2,
                    "maxItems": 4
                }
            },
            "required": ["main_research_question", "methodology_summary",
                         "data_description", "key_findings", "contribution"],
            "additionalProperties": false
        }
```

```
    }
}
```

A.4 Combined Multi-Task Applications

These schemas combine classification, retrieval, and generation within a single API call.

A.4.1 Comprehensive Earnings Call Analysis

This schema classifies management outlook, extracts financial guidance, and generates structured summaries from earnings call transcripts.

LISTING 24. **Multi-Task Earnings Call Analysis**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "analyze_earnings_call",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                # CLASSIFICATION: Management outlook
                "outlook_classification": {
                    "type": "string",
                    "enum": ["optimistic", "pessimistic", "cautious", "neutral"],
                    "description": "Overall management outlook on future performance"
                },
                # INFORMATION RETRIEVAL: Specific guidance
                "guidance": {
                    "type": "object",
                    "properties": {
                        "revenue_guidance": {
                            "type": "number",
                            "description": "Revenue guidance in millions. Null if not provided."
                        },
                        "margin_forecast": {
                            "type": "number",
```

```
                            "description": "Operating margin forecast (percentage)"
                        },
                        "capex_plans": {
                            "type": "number",
                            "description": "Planned capital expenditure in millions"
                        },
                        "guidance_period": {
                            "type": "string",
                            "description": "Period for guidance (e.g., 'Q3 2024', 'FY 2024')"
                        }
                    },
                    "required": ["guidance_period"],
                    "additionalProperties": false
                },
                # GENERATION: Structured summary
                "key_takeaways": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "Most important points for investors (3-5 items)",
                    "minItems": 3,
                    "maxItems": 5
                },
                "risk_factors_mentioned": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "Key risks or challenges discussed",
                    "minItems": 1
                },
                "confidence": {
                    "type": "number",
                    "description": "Overall confidence in the analysis (0 to 1)"
                }
            },
            "required": ["outlook_classification", "guidance", "key_takeaways",
                         "risk_factors_mentioned", "confidence"],
            "additionalProperties": false
        }
    }
}
```

A.4.2 Central Bank Communication Comprehensive Analysis

This schema classifies policy stance, extracts economic projections, and generates structured summaries from central bank communications.

LISTING 25. **Comprehensive Central Bank Analysis**

```
response_format = {
    "type": "json_schema",
    "json_schema": {
        "name": "analyze_central_bank_communication",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
                # CLASSIFICATION: Policy stance
                "policy_stance": {
                    "type": "string",
                    "enum": ["hawkish", "dovish", "neutral"],
                    "description": "Overall monetary policy stance"
                },
                # INFORMATION RETRIEVAL: Economic projections
                "economic_projections": {
                    "type": "object",
                    "properties": {
                        "policy_rate": {
                            "type": "number",
                            "description": "Current or projected policy rate (%)"
                        },
                        "inflation_target": {
                            "type": "number",
                            "description": "Stated inflation target (%)"
                        },
                        "gdp_forecast": {
                            "type": "number",
                            "description": "GDP growth forecast (%)"
                        },
                        "forecast_year": {
                            "type": "integer",
                            "description": "Year for projections"
                        }
```

```
                    },
                    "required": ["forecast_year"],
                    "additionalProperties": false
                },
                # GENERATION: Structured summary
                "summary": {
                    "type": "object",
                    "properties": {
                        "main_message": {
                            "type": "string",
                            "description": "One-sentence summary of key message"
                        },
                        "policy_actions": {
                            "type": "array",
                            "items": {"type": "string"},
                            "description": "Specific policy actions announced or discussed"
                        },
                        "forward_guidance": {
                            "type": "string",
                            "description": "Guidance on future policy path (max 100 words)"
                        },
                        "key_concerns": {
                            "type": "array",
                            "items": {"type": "string"},
                            "description": "Primary economic concerns mentioned"
                        }
                    },
                    "required": ["main_message", "policy_actions", "key_concerns"],
                    "additionalProperties": false
                },
                "confidence": {
                    "type": "number",
                    "description": "Confidence score (0 to 1)"
                }
            },
            "required": ["policy_stance", "economic_projections", "summary", "confidence"],
            "additionalProperties": false
        }
    }
}
```